

Secure Ticketing & Reservation API (Spring Boot 4, Java 21+)

Scenario

You are building an event platform. Organizers create and publish events, and customers reserve seats. Overselling must not occur, idempotent requests must be supported, and JWT-based authentication with role-based authorization must be applied. The core flow is simple enough to complete in a few hours, but the case allows evaluation of architecture, security, and design quality.

Roles

- ADMIN: Full access to all resources.
- ORGANIZER: Full access to owned events.
- CUSTOMER: Reservation operations.

Entities

- User: id, email(unique), passwordHash, roles, createdAt, lastLoginAt
- Event: id, ownerId, title, venue, startsAt, endsAt, capacity, published(boolean), version
- Reservation: id, eventId, userId, status(PENDING/CONFIRMED/CANCELLED), seats, createdAt
- IdempotencyKey: id, key, endpoint, requestHash, responseHash, status, createdAt, ttl
- AuditLog: id, actorId, action, resourceType, resourceId, ip, userAgent, createdAt

Core API Endpoints

Authentication & Authorization

POST /api/auth/register — Registers a new user.

POST /api/auth/login — Issues JWT access & refresh tokens.

POST /api/auth/refresh — Refresh-token flow.

Event Management

POST /api/events — Creates a draft event owned by the authenticated organizer/admin.

PUT /api/events/{id} — Updates an event if caller is owner or admin.

POST /api/events/{id}/publish — Publishes a draft event.

GET /api/events?ownerId=... — Lists events with optional filtering.

Reservations

POST /api/events/{id}/reservations — Creates a PENDING reservation, requires Idempotency-Key.

POST /api/reservations/{id}/confirm — Confirms a reservation.

POST /api/reservations/{id}/cancel — Cancels a reservation.

Discovery

GET /api/events/public?from=...&to=...&q=... — Retrieves published events with filtering.

Testing Expectations

- Unit tests for core logic and security rules.
- Integration tests with H2; concurrency tests using multiple threads.
- Security tests for role-based access.
- Coverage target ~70% on logic classes (not mandatory).

README Expectations

- Setup instructions (Maven/Gradle), optional docker-compose.
- Auth flow description with sample JWT.
- Architectural decisions (ADR or reasoning section).
- Seed users.
- OpenAPI spec.

Evaluation Rubric

Category	Expected	Points
Functionality	CRUD + publish + reserve + confirm + cancel	15
Security	JWT, BCrypt, role-based auth, rate limit, audit	15
Data Consistency	No oversell, proper idempotency	15
Tests	Unit + integration + concurrency + security	15
Architecture & Design	Layered design, exception model	15
Observability & Docs	Actuator, OpenAPI, strong README	10
Code Quality	Clean code, minimal static analysis warnings	15